How Much Tensor Structure Should a Transformer FFN Have?

Routed Block-CP Feedforward Networks Between SwiGLU and Dense Tucker

Aniket Deshpande
University of Illinois Urbana-Champaign
aniket4@illinois.edu

June 2026

Abstract

[Abstract placeholder.]

1 Introduction

[Introduction placeholder.]

2 The routed tensor view of gated FFNs

Setup. At a fixed token position, let $x \in \mathbb{R}^d$ be the residual-stream vector entering the feedforward block. The SwiGLU FFN (Shazeer, 2020) computes

$$y = U^\top h, \qquad h = (W^\top x) \odot \mathrm{SiLU}(G^\top x), \tag{1}$$

with $W, G \in \mathbb{R}^{d\times m}$ and $U \in \mathbb{R}^{m\times d}$. Writing w_j, g_j for the j-th columns of W, G and u_j for the j-th column of $U^\top$, the j-th hidden coordinate is $h_j(x) = (w_j^\top x)\,\mathrm{SiLU}(g_j^\top x)$.

2.1 SwiGLU is an exactly routed CP tensor field

Each SwiGLU channel multiplies two scalar projections of the residual stream. The SiLU nonlinearity factorizes exactly, $\mathrm{SiLU}(z) = z\,\sigma(z)$ with $\sigma(z) = (1+e^{-z})^{-1}$, so

$$h_j(x) \;=\; \underbrace{(w_j^\top x)(g_j^\top x)}_{\text{rank-one bilinear interaction}} \cdot \underbrace{\sigma(g_j^\top x)}_{\alpha_j(x)\,\in\,(0,1)}. \tag{2}$$

Defining the input-dependent interaction tensor $A(x) \in \mathbb{R}^{d\times d\times d}$ by $y_o = \sum_{a,b} A_{oab}(x)\,x_a x_b$, substituting (2) into (1) gives

$$A(x) \;=\; \sum_{j=1}^{m} \alpha_j(x)\; u_j \otimes w_j \otimes g_j, \qquad \alpha_j(x) = \sigma(g_j^\top x). \tag{3}$$

This is a canonical polyadic (CP) decomposition (Kolda and Bader, 2009) of $A(x)$ with *shared, input-independent factors* and *input-dependent routing weights* $\alpha_j(x)$. The decomposition is exact and holds pointwise in x; no approximation is involved. We call $u_j \otimes w_j \otimes g_j$ the j-th *interaction*

atom (what interaction is available) and $\alpha_j(x)$ its *route* (when it is used). The bilinear $\alpha \equiv 1$ limit of this picture is the bilinear MLP analyzed by Pearce et al. (2025); the routed form keeps the gate inside the structural description rather than discarding it.

Viewed as a Tucker decomposition with latent dimensions $r = s = m$ and factor matrices $(P, Q, R) = (W, G, U^\top)$, the CP form (3) corresponds to a *superdiagonal core*: $C_{oij} = \delta_{oi}\delta_{ij}$. This restriction cuts both ways. It atomizes the layer — every hidden index is one (atom, route) pair, and CP decompositions are essentially unique under mild conditions (Kruskal, 1977), so the atoms are identifiable objects. But it also means one route controls exactly one rank-one interaction: feature w_i never interacts with gate g_j for $i \neq j$.

2.2 Dense Tucker and the per-route interaction matrix

The unconstrained relaxation drops the superdiagonal restriction. A *Tucker-core FFN* with latent projections $P, Q \in \mathbb{R}^{d\times r}$, output basis $R \in \mathbb{R}^{d\times s}$, and learned core $C \in \mathbb{R}^{s\times r\times r}$ computes $p = P^\top x$, $q = Q^\top x$,

$$z_o = \sum_{i,j=1}^{r} C_{oij}\, p_i\, \mathrm{SiLU}(q_j), \qquad y = Rz. \tag{4}$$

Grouping by gate index exposes the structure this paper is about:

$$y(x) = \sum_{j=1}^{r} V_j\, p\, \mathrm{SiLU}(q_j), \qquad V_j := R\, C^{(j)} \in \mathbb{R}^{d\times r}, \quad C^{(j)}_{oi} = C_{oij}. \tag{5}$$

Each latent gate q_j routes a *matrix* V_j of output-by-main interactions. SwiGLU is the corner case $\operatorname{rank} V_j = 1$ for all j; a generic Tucker core has $\operatorname{rank} V_j = \min(r, s)$.

Two properties make the dense core suspicious as an interpretability target. First, *gauge freedom*: for any invertible $M \in \mathbb{R}^{r\times r}$, replacing $P \mapsto PM^{-\top}$ and $C^{(j)} \mapsto C^{(j)}M^\top$ for all j leaves the function unchanged, so individual latent directions p_i and individual core entries carry no intrinsic meaning — only the gate directions $Q_{:j}$ and the subspaces $\operatorname{range}(V_j)$ are well defined. Second, the core costs sr^2 parameters; at the configurations we study it absorbs over 90% of the block's budget.

The aligned-width theorem, reinterpreted. Prior work on this codebase proved an exact separation: if $[P\ Q]$ has full column rank, an *aligned* SwiGLU (units restricted to main vectors in $\operatorname{span}(P)$ and gate vectors from the columns of Q) that exactly realizes (5) must have width $m \geq \sum_j \operatorname{rank} V_j$, and a rank decomposition of each V_j attains the bound (Appendix A). Generically this is $r\min(r, s)$, i.e. k^2 in the balanced case — the result that originally motivated dense Tucker as "$k\times$ cheaper than SwiGLU." The reading we adopt here is different: *the theorem says the operative variable is the per-route rank* $\operatorname{rank} V_j$, not the presence of a dense core. The constructive half of the proof — realize each V_j as a sum of $\operatorname{rank} V_j$ rank-one terms sharing the same gate — is itself an architecture. That architecture is the subject of this paper.

2.3 LL1/block-CP: one route, one rank-L block

A third-order tensor admits a *decomposition in multilinear rank-$(L, L, 1)$ terms* (LL1; a structured block-term decomposition) if it can be written $T = \sum_{b=1}^{B} (A_b B_b^\top) \otimes c_b$: each term is rank-$L$ in two modes and rank-one in the third (De Lathauwer, 2008; Vervliet et al., 2016). Taking the *gate mode* as the rank-one mode yields the FFN we study. The **LL1 FFN** with B blocks of rank L has, per

block, a gate direction $g_b \in \mathbb{R}^d$, a main factor $A_b \in \mathbb{R}^{d\times L}$, and an output factor $U_b \in \mathbb{R}^{d\times L}$:

$$y(x) \;=\; \sum_{b=1}^{B} U_b\big(A_b^\top x\big)\,\mathrm{SiLU}(g_b^\top x). \tag{6}$$

Expanding into atoms, $y(x) = \sum_b \sum_{l=1}^{L} u_{b,l}(a_{b,l}^\top x)\,\mathrm{SiLU}(g_b^\top x)$: *grouped CP*, L rank-one atoms sharing one route. The interaction tensor is the routed LL1 tensor $A(x) = \sum_b \alpha_b(x)\,(U_b A_b^\top) \otimes g_b$, and the per-route matrix is $V_b = U_b A_b^\top$ with $\operatorname{rank} V_b \le L$ *by construction*. The aligned-width theorem's control variable has become a hyperparameter.

The family nests both endpoints exactly. At $L = 1$, $B = m$, (6) *is* SwiGLU (verified to machine precision in our implementation); a single block with $L = r$ is a (one-gate) dense slice; and an LL1 FFN equals a Tucker FFN whose core is block-superdiagonal. LL1 decompositions also retain essential-uniqueness guarantees under mild conditions (De Lathauwer, 2008, 2011), unlike the gauge-free dense core.

What the parameter budget buys. The comparison that matters is at matched parameters:

$$N_{\mathrm{SwiGLU}} = 3dm, \qquad N_{\mathrm{LL1}} = dB(2L+1), \qquad N_{\mathrm{Tucker}} = d(2r+s) + sr^2. \tag{7}$$

At a fixed budget N, SwiGLU affords $m = N/3d$ atoms, each with a private route; LL1 affords $BL = \frac{3L}{2L+1} \cdot \frac{N}{3d}$ atoms — up to 1.5× more — but only $B = \frac{3}{2L+1} \cdot \frac{N}{3d}$ routes. *LL1 trades routing diversity for interaction atoms.* An equivalent statement: an LL1(B, L) block is exactly a width-BL SwiGLU whose gate vectors are tied in groups of L. Which side of the trade wins is an empirical question about what trained FFNs actually need, and it is the question our experiments are designed to answer. A prior observation sharpens the stakes: in a dense Tucker LM trained from scratch, the learned per-gate stable rank $\|V_j\|_F^2/\|V_j\|_{\mathrm{op}}^2$ concentrates around 4 — far below the generic value r, far above the SwiGLU value 1. The trained dense core already behaves like an LL1 model; LL1 makes that behavior an explicit, cheaper, and identifiable parameterization.

3 Experiments

3.1 Synthetic teacher–student recovery

[exp18 placeholder]

3.2 Matched-budget language modeling

[exp11/sprint_lm placeholder]

3.3 Interpretability proxies

[exp19/exp22 placeholder]

3.4 Circuit-level pilot: induction

[exp20 placeholder]

4 Discussion

[placeholder: what is established / plausible / open; limitations; next experiment.]

5 Related work

Gated FFNs and bilinear MLPs. GLU-family FFNs outperform ReLU/GELU MLPs at matched scale and are standard in modern LMs (Shazeer, 2020). Pearce et al. (2025) study bilinear MLPs — GLUs with the element-wise nonlinearity removed — which are exactly third-order tensors, and show that eigendecomposition of those tensors yields interpretable low-rank structure; Dooms and Wilhelm (2024) make the architectural case. Our object differs in keeping the gate: SwiGLU is exactly a *routed* CP tensor field whose routing is input-dependent, and prior experiments on this codebase show the routing is load-bearing (constant-α ablations degrade pretrained Qwen models by four to six orders of magnitude in perplexity), so the bilinear picture alone does not describe trained SwiGLU models. Jayakumar et al. (2020) situate gating within multiplicative interactions broadly; we study the specific tensor decompositions a gated FFN admits.

Tensor decompositions in neural networks. CP, Tucker, and block-term decompositions are classical (Kolda and Bader, 2009; De Lathauwer, 2008). In deep learning they are most often used for *compression* of trained weights (Novikov et al., 2015) or for expressivity analyses of idealized architectures (Cohen et al., 2016); factorization machines use low-rank multiplicative atoms for feature interactions (Rendle, 2010); Loconte et al. (2024) connect tensor factorizations to probabilistic circuits. Our use is different: the decomposition is not applied post hoc to a weight tensor — the FFN *is* the decomposition, and we ask which member of the CP→LL1→Tucker family is the right parameterization to train.

Sparse dictionaries and weight-based interpretability. Sparse autoencoders and transcoders decompose activations into feature directions (Bricken et al., 2023; Dunefsky et al., 2024; Paulo et al., 2025). The routed-CP/LL1 view is complementary: atoms and routes are *architectural* objects fixed in the weights, with the input-dependence isolated in scalar routing coefficients — the same invariant-dictionary + input-dependent-coefficients factorization transcoders recover empirically.

Structured matrices for efficiency. Monarch and butterfly factorizations replace dense projections with products of block-diagonal or sparse factors (Dao et al., 2022, 2019); LoRA demonstrates that rank constraints are effective in transformers (Hu et al., 2022). These address the *factor matrices*; our axis is the *latent core* connecting them. The two are orthogonal and composable.

Circuit-level evaluation. Induction heads are the canonical recognizable circuit (Elhage et al., 2021; Olsson et al., 2022); we use a minimal induction task to ask whether FFN tensor structure changes the emergence or mechanism of an attention-side circuit.

References

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

Nadav Cohen, Or Sharir, and Amnon Shashua. On the expressive power of deep learning: A tensor analysis. In *Conference on Learning Theory*, 2016.

Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra, and Christopher Ré. Learning fast algorithms for linear transforms using butterfly factorizations. In *International Conference on Machine Learning*, 2019.

Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *International Conference on Machine Learning*, 2022.

Lieven De Lathauwer. Decompositions of a higher-order tensor in block terms—part II: Definitions and uniqueness. *SIAM Journal on Matrix Analysis and Applications*, 30(3):1033–1066, 2008.

Lieven De Lathauwer. Blind separation of exponential polynomials and the decomposition of a tensor in rank-$(l_r, l_r, 1)$ terms. *SIAM Journal on Matrix Analysis and Applications*, 32(4):1451–1474, 2011.

Thomas Dooms and Daniel Wilhelm. Weight-based decomposition: A case for bilinear MLPs. *arXiv preprint arXiv:2406.03947*, 2024.

Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. *arXiv preprint arXiv:2406.11944*, 2024.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Siddhant M. Jayakumar, Wojciech M. Czarnecki, Jacob Menick, Jonathan Schwarz, Jack Rae, Simon Osindero, Yee Whye Teh, Tim Harley, and Razvan Pascanu. Multiplicative interactions and where to find them. In *International Conference on Learning Representations*, 2020.

Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51 (3):455–500, 2009.

Joseph B. Kruskal. Three-way arrays: Rank and uniqueness of trilinear decompositions, with application to arithmetic complexity and statistics. *Linear Algebra and its Applications*, 18(2): 95–138, 1977.

Lorenzo Loconte, Antonio Mari, Gennaro Gala, Robert Peharz, Cassio de Campos, Erik Quaeghebeur, Gennaro Vessio, and Antonio Vergari. What is the relationship between tensor factorizations and circuits (and how can we exploit it)? *arXiv preprint arXiv:2409.07953*, 2024.

Alexander Novikov, Dmitry Podoprikhin, Anton Osokin, and Dmitry Vetrov. Tensorizing neural networks. In *Advances in Neural Information Processing Systems*, 2015.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

Gonçalo Paulo, Stepan Shabalin, and Nora Belrose. Transcoders beat sparse autoencoders for interpretability. *arXiv preprint arXiv:2501.18823*, 2025.

Michael T. Pearce, Thomas Dooms, Alice Rigg, Jose M. Oramas, and Lee Sharkey. Bilinear MLPs enable weight-based mechanistic interpretability. In *International Conference on Learning Representations*, 2025.

Steffen Rendle. Factorization machines. In *IEEE International Conference on Data Mining*, 2010.

Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Nico Vervliet, Otto Debals, Laurent Sorber, Marc Van Barel, and Lieven De Lathauwer. Tensorlab 3.0. *Available online at https://www.tensorlab.net*, 2016.

A The aligned-width theorem

This appendix restates, with proof, the separation theorem from the earlier draft accompanying this codebase, and records its constructive half as the LL1 architecture. Throughout, $\phi = \mathrm{SiLU}$; the only properties used are $\phi(0) = 0$ and $\phi(1) \neq 0$.

Lemma 1 (SwiGLU recovery)**.** *Setting $r = s = m$, $P = W$, $Q = G$, $R = U^\top$, and $C_{oij} = \delta_{oi}\delta_{ij}$ in (4) reduces the Tucker-core FFN to the SwiGLU map (1).*

Proof. With the superdiagonal core, (4) collapses to $z_o = p_o\,\phi(q_o)$, so $z = (P^\top x) \odot \phi(Q^\top x)$, which is h from (1); then $y = Rz = U^\top h$. □

Definition 1 (Aligned SwiGLU representation)**.** *Given latent matrices $P, Q \in \mathbb{R}^{d\times r}$, an* aligned width-$m$ SwiGLU representation *is a map*

$$\hat{y}(x) = \sum_{j=1}^{r} \sum_{\nu=1}^{k_j} u_{j\nu}\,(a_{j\nu}^\top p)\,\phi(q_j), \qquad \textstyle\sum_j k_j = m,$$

with $u_{j\nu} \in \mathbb{R}^d$, $a_{j\nu} \in \mathbb{R}^r$, $p = P^\top x$, $q = Q^\top x$. Each unit uses a main vector in $\mathrm{span}(P)$ *and one of the gate vectors $Q_{:j}$.*

Theorem 1 (Diagonal bottleneck)**.** *Assume $[P\ Q] \in \mathbb{R}^{d\times 2r}$ has full column rank. The minimum width of an aligned SwiGLU representation exactly realizing a Tucker-core FFN with per-gate coefficient matrices $\{V_j\}_{j=1}^r$ is $m_{\min} = \sum_{j=1}^r \operatorname{rank} V_j$.*

Proof. Lower bound. By the rank assumption, $x \mapsto (p, q)$ is surjective onto $\mathbb{R}^r \times \mathbb{R}^r$, so an exact representation forces $\sum_j (V_j - \hat{V}_j)\,p\,\phi(q_j) = 0$ for all (p, q), where $\hat{V}_j = \sum_\nu u_{j\nu}a_{j\nu}^\top$ has rank at most k_j. Fix j_0 and set $q_j = \delta_{jj_0}$; since $\phi(0) = 0$ and $\phi(1) \neq 0$, only the j_0 term survives and $V_{j_0} = \hat{V}_{j_0}$. Hence $k_j \geq \operatorname{rank} V_j$ for every j, and $m = \sum_j k_j \geq \sum_j \operatorname{rank} V_j$.

Tightness. Let $V_j = \sum_{\nu=1}^{\rho_j} u_{j\nu}a_{j\nu}^\top$ be any rank decomposition, $\rho_j = \operatorname{rank} V_j$. Then $V_j\,p\,\phi(q_j) = \sum_\nu u_{j\nu}\big((Pa_{j\nu})^\top x\big)\,\phi(Q_{:j}^\top x)$ is a sum of ρ_j aligned units sharing gate $Q_{:j}$. □

Corollary 1 (Generic separation)**.** *If R has full column rank and each slice $C^{(j)}$ is generic, then* $\operatorname{rank} V_j = \min(r, s)$ *for all j and $m_{\min} = r\min(r, s)$; in the balanced case $r = s = k$, $m_{\min} = k^2$.*

Remark 1 (The constructive half is LL1)**.** *The tightness construction — ρ_j rank-one units sharing the gate $Q_{:j}$ — is precisely one block of the LL1 FFN (6) with $L = \rho_j$. Theorem 1 therefore does not say "dense cores beat SwiGLU"; it says the cost of simulating a routed tensor map is the sum of its per-route ranks, and an architecture that fixes those ranks directly realizes the map at exactly that cost.*

B Experimental details

[hyperparameters, configs, hardware.]